PYTHON PROGRAMMING

PYTHON

Functions & Methods

Reference Guide

A practical guide to built-in functions, data types, methods, operators and everyday Python tools.

STUDENT EDITION · PYTHON 3

A learning and revision reference

PYTHON
Functions & Methods Reference Guide

A practical, student-friendly reference to built-in functions, data-type methods, operators and everyday programming tools.

GUIDE FEATURE	WHAT IT COVERS
`Foundations first`	Numbers, types, conversions and basic input/output
`Data types`	Strings, lists, tuples, sets and dictionaries
`Programming helpers`	Built-ins, iteration, operators and math utilities
`Practical topics`	Random values, files and exception handling
`Learning support`	Examples, expected outputs and common-mistake callouts

Examples: Written for Python 3.11 and suitable for modern Python 3 versions unless otherwise noted.

How to use this guide: Find the relevant category, read the purpose and example, then run the code in a Python editor.

This is a practical reference guide, not an exhaustive catalogue of every function in the Python standard library.

Contents

1. Foundations: functions, methods and operators . . . 4

2. Numeric types and common operations . . . 5

3. Type checking and conversion . . . 6

4. Input, output and basic utilities . . . 7

5. String methods . . . 8

6. List methods . . . 10

7. Tuple methods and operations . . . 11

8. Set methods and set operations . . . 12

9. Dictionary methods . . . 13

10. Common built-in functions . . . 14

11. Operators . . . 15

12. Math module and numeric helpers . . . 16

13. Random module . . . 17

14. File handling . . . 18

15. Exception handling . . . 19

16. Quick reference and common mistakes . . . 20

Page numbers are generated from the final layout. Use the PDF bookmarks panel to jump directly to major sections.

1. Foundations: functions, methods and operators

Python offers built-in functions, methods attached to objects, operators, and statements. They are related, but they are not the same thing.

Kind	What it means	Example	Result
`Built-in function`	Called by name; often takes an object as an argument.	`len('Python')`	6
`Method`	Called on an object using dot notation.	`'python'.upper()`	'PYTHON'
`Operator`	Symbol or keyword that performs an operation.	`3 in [1, 2, 3]`	True
`Statement`	Instruction that forms part of program flow.	`del items[0]`	Deletes an item

Syntax patterns

```
# Built-in function
len(my_list)

# Method
my_list.append(10)

# Operator
10 in my_list

# Statement (not a function or method)
del my_list[0]
```

Remember: A method may modify an object or return a result. For example, list.append() changes the list and returns None; str.upper() returns a new string.

2. Numeric types and common operations

Python's main numeric types are int (whole numbers), float (floating-point numbers) and complex (numbers with real and imaginary parts).

Function / method	Purpose	Example	Output / notes
`int`	Whole-number values.	`type(42)`	<class 'int'>
`float`	Numbers with a fractional part.	`type(3.5)`	<class 'float'>
`complex`	Real and imaginary components.	`complex(2, 3)`	(2+3j)
`abs(x)`	Absolute value.	`abs(-7)`	7
`round(x, ndigits)`	Rounds to a chosen precision.	`round(3.14159, 2)`	3.14
`pow(x, y)`	Raises x to the power y.	`pow(2, 3)`	8
`divmod(a, b)`	Returns quotient and remainder.	`divmod(17, 5)`	(3, 2)
`int.bit_length()`	Bits needed for the integer's magnitude.	`(10).bit_length()`	4
`float.is_integer()`	Checks if a float has no fractional part.	`(5.0).is_integer()`	True
`float.as_integer_ratio()`	Exact numerator/denominator ratio.	`(0.5).as_integer_ratio()`	(1, 2)
`bin(x)`	Integer represented as binary text.	`bin(10)`	'0b1010'
`oct(x)`	Integer represented as octal text.	`oct(8)`	'0o10'
`hex(x)`	Integer represented as hexadecimal text.	`hex(255)`	'0xff'

Worked example

```
price = 19.99
quantity = 3
total = price * quantity

print(total)          # 59.97
print(round(total, 1))# 60.0
print(17 // 5)        # 3: floor division
print(17 % 5)         # 2: remainder
print(2 ** 4)         # 16: exponentiation
```

Common mistake: Floating-point calculations can have tiny precision differences. For exact decimal arithmetic in financial applications, consider the decimal.Decimal type.

3. Type checking and conversion

Conversion functions create values of another type when the input is compatible. Invalid text can raise ValueError.

Function / method	Purpose	Example	Output / notes
type(x)	Returns an object's type.	`type('hi')`	<class 'str'>
isinstance(x, T)	Checks whether x is an instance of T.	`isinstance(5, int)`	True
int(x)	Converts compatible values to an integer.	`int('25')`	25
float(x)	Converts compatible values to a float.	`float('3.5')`	3.5
str(x)	Returns a string representation.	`str(100)`	'100'
bool(x)	Applies truth-value rules.	`bool(0)`	False
complex(a, b)	Creates a complex number.	`complex(2, 3)`	(2+3j)
list(iterable)	Creates a list.	`list('abc')`	['a', 'b', 'c']
tuple(iterable)	Creates a tuple.	`tuple([1, 2])`	(1, 2)
set(iterable)	Creates a set of unique hashable values.	`set([1, 1, 2])`	{1, 2}
dict(...)	Creates a dictionary.	`dict(a=1, b=2)`	{'a': 1, 'b': 2}
chr(n)	Converts a Unicode code point to a character.	`chr(65)`	'A'
ord(c)	Returns a character's Unicode code point.	`ord('A')`	65
repr(x)	Returns a developer-oriented representation.	`repr('hi')`	"'hi'"
format(x, spec)	Formats a value using a format specification.	`format(3.14159, '.2f')`	'3.14'

```
raw_values = ["10", "20", "30"]
numbers = list(map(int, raw_values))

print(numbers)       # [10, 20, 30]
print(tuple(numbers))# (10, 20, 30)
print(int(float("3.5"))) # 3
```

Common mistake: int('3.5') raises ValueError. If truncating the decimal part is intended, int(float('3.5')) returns 3. Conversion should match the task's requirements.

4. Input, output and basic utilities

These are among the first functions used in small Python programs.

Function / method	Purpose	Example	Output / notes
`print(*objects)`	Displays values.	`print('Hi', 5)`	Hi 5
`input(prompt)`	Reads user input; always returns text.	`name = input('Name: ')`	Waits for typing
`len(obj)`	Counts characters or items.	`len('hello')`	5
`help(obj)`	Shows documentation in interactive Python.	`help(len)`	Help text
`dir(obj)`	Lists names available on an object.	`dir('abc')`	Attribute-name list
`callable(obj)`	Checks whether an object can be called.	`callable(len)`	True
`id(obj)`	Returns an identity integer for an object's lifetime.	`id(x)`	Value varies
`open(file, mode)`	Opens a file and returns a file object.	`open('notes.txt', 'r')`	File object; file must exist
`exit()`	Interactive-session exit helper.	`exit()`	Ends interactive session

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print(f"Hello {name}. Next year you will be {age + 1}.")

# Example input:
# Enter your name: Anu
# Enter your age: 20
# Output:
# Hello Anu. Next year you will be 21.
```

Safety note: Avoid eval() on untrusted text: it can execute arbitrary Python expressions. For program termination in scripts, sys.exit() is generally clearer than the interactive exit() helper.

5. String methods

Strings are ordered sequences of characters and are immutable: methods return new strings rather than changing the original.

Case conversion

Function / method	Purpose	Example	Output / notes
`upper()`	Converts letters to uppercase.	`'python'.upper()`	'PYTHON'
`lower()`	Converts letters to lowercase.	`'PYTHON'.lower()`	'python'
`title()`	Uppercases the start of each word.	`'hello world'.title()`	'Hello World'
`capitalize()`	Uppercases first character; lowercases the rest.	`'pYTHON'.capitalize()`	'Python'
`swapcase()`	Swaps uppercase and lowercase.	`'PyThon'.swapcase()`	'pYtHON'
`casefold()`	Strong lowercase form for caseless matching.	`'Straße'.casefold()`	'strasse'

Cleaning and replacing

Function / method	Purpose	Example	Output / notes
`strip()`	Removes whitespace at both ends.	`' hi '.strip()`	'hi'
`lstrip() / rstrip()`	Removes whitespace on left / right.	`' hi '.lstrip()`	'hi '
`replace(old, new, count)`	Replaces matching text.	`'a-b-c'.replace('-', '/')`	'a/b/c'
`removeprefix(p)`	Removes matching prefix if present.	`'unhappy'.removeprefix('un')`	'happy'
`removesuffix(s)`	Removes matching suffix if present.	`'file.txt'.removesuffix('.txt')`	'file'

Splitting and joining

Function / method	Purpose	Example	Output / notes
`split(sep=None)`	Splits text into a list.	`'a,b,c'.split(',')`	['a', 'b', 'c']
`rsplit(sep, maxsplit)`	Splits from the right.	`'a-b-c'.rsplit('-', 1)`	['a-b', 'c']
`splitlines()`	Splits at line boundaries.	`'a\\nb'.splitlines()`	['a', 'b']
`join(iterable)`	Joins strings with this separator.	`'-'.join(['a', 'b'])`	'a-b'

Searching and checking

Function / method	Purpose	Example	Output / notes
`find(sub) / rfind(sub)`	First / last position, or -1.	`'hello'.rfind('l')`	3
`index(sub) / rindex(sub)`	First / last position; error if absent.	`'hello'.index('e')`	1
`count(sub)`	Counts non-overlapping occurrences.	`'banana'.count('a')`	3
`startswith(p) / endswith(s)`	Checks prefix / suffix.	`'data.csv'.endswith('.csv')`	True
`isalpha()`	True if all characters are letters and non-empty.	`'abc'.isalpha()`	True
`isdigit()`	True if all characters are digits and non-empty.	`'123'.isdigit()`	True
`isdecimal()`	True if all characters are decimal digits.	`'123'.isdecimal()`	True
`isalnum()`	True if letters/digits only and non-empty.	`'abc123'.isalnum()`	True
`isspace()`	True if all characters are whitespace and non-empty.	`' '.isspace()`	True
`islower() / isupper()`	Checks letter case.	`'ABC'.isupper()`	True
`istitle()`	Checks title-case pattern.	`'Hello World'.istitle()`	True

Formatting and alignment

Function / method	Purpose	Example	Output / notes
`format()`	Fills format placeholders.	`'Hello, {}!'.format('Sam')`	'Hello, Sam!'
`center(width, fill)`	Centers text in a field.	`'hi'.center(6, '*')`	'**hi**'
`ljust() / rjust()`	Left / right aligns with padding.	`'hi'.rjust(5, '.')`	'...hi'
`zfill(width)`	Pads on the left with zeroes.	`'42'.zfill(5)`	'00042'

```
text = "  Python is FUN  "
clean = text.strip()

print(clean.lower())                # python is fun
print(clean.replace("FUN", "useful"))# Python is useful
print("red,green,blue".split(","))   # ['red', 'green', 'blue']
print(" | ".join(["red", "green"])) # red | green
```

Common mistake: text.upper() alone does not change text. Assign the result if you want to keep it: text = text.upper().

6. List methods

Lists are ordered and mutable. The methods below are grouped by the kind of operation they perform.

Adding

Function / method	Purpose	Example	Output / notes
`append(item)`	Adds one item at the end.	`x = [1, 2]; x.append(3)`	x is [1, 2, 3]
`extend(iterable)`	Adds each item from an iterable.	`x = [1]; x.extend([2, 3])`	x is [1, 2, 3]
`insert(index, item)`	Inserts before the given index.	`x = [1, 3]; x.insert(1, 2)`	x is [1, 2, 3]

Removing

Function / method	Purpose	Example	Output / notes
`remove(value)`	Removes first matching value.	`x = [1, 2, 3]; x.remove(2)`	x is [1, 3]
`pop(index=-1)`	Removes and returns an item.	`x = [1, 2, 3]; x.pop()`	Returns 3; x is [1, 2]
`clear()`	Removes all items.	`x = [1, 2]; x.clear()`	x is []
`del (statement)`	Deletes an item or slice by position.	`x = [1, 2, 3]; del x[0]`	x is [2, 3]

Ordering and copying

Function / method	Purpose	Example	Output / notes
`sort(key=None, reverse=False)`	Sorts the original list in place.	`x = [3, 1, 2]; x.sort()`	x is [1, 2, 3]; returns None
`reverse()`	Reverses original order in place.	`x = [1, 2, 3]; x.reverse()`	x is [3, 2, 1]; returns None
`copy()`	Creates a shallow copy.	`y = x.copy()`	Nested mutable items may be shared
`sorted(x) (built-in)`	Returns a new sorted list.	`sorted([3, 1, 2])`	[1, 2, 3]

Searching

Function / method	Purpose	Example	Output / notes
`index(value)`	Returns first matching position.	`[10, 20, 30].index(20)`	1
`count(value)`	Counts matching values.	`[1, 2, 2, 3].count(2)`	2
`in / not in (operator)`	Tests membership.	`3 in [1, 2, 3]`	True

Common mistake: append(), sort() and reverse() return None. Do not write x = x.sort(). Use sorted(x) when you need a new sorted list.

7. Tuple methods and operations

Tuples are ordered and immutable. They have only two methods of their own; built-in functions such as len() and sum() also work with tuples.

Function / method	Purpose	Example	Output / notes
`count(value)`	Counts occurrences.	`(1, 2, 2).count(2)`	2
`index(value)`	Finds first matching position.	`(5, 6, 7).index(6)`	1
`len(T)`	Counts items.	`len((1, 2, 3))`	3
`sorted(T)`	Returns a sorted list.	`sorted((3, 1, 2))`	[1, 2, 3]
`tuple(iterable)`	Creates a tuple.	`tuple([1, 2])`	(1, 2)
`min(T) / max(T)`	Finds smallest / largest item.	`max((4, 9, 2))`	9
`sum(T)`	Adds numeric items.	`sum((2, 3, 4))`	9
`reversed(T)`	Returns a reverse iterator.	`tuple(reversed((1, 2, 3)))`	(3, 2, 1)

```
T = (10, 20, 10, 30)
print(T.count(10))   # 2
print(T.index(20))   # 1
print(len(T))        # 4
```

Common mistake: (5) is the integer 5. A one-item tuple needs a comma: (5,).

8. Set methods and set operations

Sets contain unique hashable values and are useful for removing duplicates and comparing groups of values.

Adding and removing

Function / method	Purpose	Example	Output / notes
`add(item)`	Adds one element.	`s = {1, 2}; s.add(3)`	s contains 1, 2, 3
`update(iterable)`	Adds elements from iterable(s).	`s = {1}; s.update([2, 3])`	s contains 1, 2, 3
`remove(item)`	Removes item; error if absent.	`s = {1, 2}; s.remove(2)`	s is {1}
`discard(item)`	Removes item if present.	`s = {1, 2}; s.discard(9)`	s unchanged
`pop()`	Removes an arbitrary element.	`s = {1, 2}; s.pop()`	Removed item is unpredictable
`clear()`	Removes all elements.	`s = {1, 2}; s.clear()`	s is set()
`copy()`	Creates a shallow copy.	`t = s.copy()`	New set

Set mathematics

Function / method	Purpose	Example	Output / notes
`union(B) / A | B`	All elements from both sets.	`{1, 2}.union({2, 3})`	{1, 2, 3}
`intersection(B) / A & B`	Elements common to both.	`{1, 2} & {2, 3}`	{2}
`difference(B) / A - B`	Elements in A but not B.	`{1, 2, 3} - {2}`	{1, 3}
`symmetric_difference(B) / A ^ B`	Elements in either but not both.	`{1, 2} ^ {2, 3}`	{1, 3}
`issubset(B)`	True if every A element is in B.	`{1}.issubset({1, 2})`	True
`issuperset(B)`	True if A contains all B elements.	`{1, 2}.issuperset({1})`	True
`isdisjoint(B)`	True if no elements are shared.	`{1}.isdisjoint({2})`	True

Common mistake: {} creates an empty dictionary, not an empty set. Use set(). Set display order is not guaranteed.

9. Dictionary methods

Dictionaries store key-value pairs. Keys are unique and must be hashable; values can be of many types.

Accessing

Function / method	Purpose	Example	Output / notes
get(key, default=None)	Gets a value without KeyError if missing.	`{'a': 1}.get('b', 0)`	0
keys()	Returns a view of keys.	`list({'a': 1, 'b': 2}.keys())`	['a', 'b']
values()	Returns a view of values.	`list({'a': 1}.values())`	[1]
items()	Returns key-value pairs.	`list({'a': 1}.items())`	[('a', 1)]

Adding, updating and removing

Function / method	Purpose	Example	Output / notes
`D[key] = value`	Adds a key or replaces its value.	`d = {'a': 1}; d['b'] = 2`	d becomes {'a': 1, 'b': 2}
update(other)	Adds or updates entries.	`d = {'a': 1}; d.update({'b': 2})`	d contains both keys
setdefault(key, default)	Returns value; inserts default if missing.	`d = {}; d.setdefault('b', 5)`	Returns 5; d becomes {'b': 5}
pop(key[, default])	Removes key and returns its value.	`{'a': 1}.pop('a')`	1
popitem()	Removes last inserted key-value pair.	`d = {'a': 1, 'b': 2}; d.popitem()`	('b', 2)
clear()	Removes all entries.	`d = {'a': 1}; d.clear()`	{}
copy()	Creates a shallow copy.	`e = d.copy()`	Nested objects may be shared
dict.fromkeys(keys, value)	Creates a dictionary from keys.	`dict.fromkeys(['a', 'b'], 0)`	{'a': 0, 'b': 0}

```
student = {"name": "Anu", "score": 85}
print(student.get("age", 0)) # 0
student["score"] = 90
student["city"] = "Pune"
print(list(student.keys())) # ['name', 'score', 'city']
```

Common mistake: d['missing'] raises KeyError. Use d.get('missing', default) when a key may not exist.

10. Common built-in functions

These functions are available without importing a module. Some apply to many data types; others work with iterables.

Function / method	Purpose	Example	Output / notes
`len(obj)`	Counts items or characters.	`len('Python')`	6
`sum(iterable)`	Adds numeric items.	`sum([2, 3, 4])`	9
`min(iterable)`	Finds smallest item.	`min([7, 2, 5])`	2
`max(iterable)`	Finds largest item.	`max([7, 2, 5])`	7
`sorted(iterable)`	Returns a new sorted list.	`sorted([3, 1, 2])`	[1, 2, 3]
`any(iterable)`	True if at least one item is truthy.	`any([0, '', 5])`	True
`all(iterable)`	True if all items are truthy.	`all([1, 2, 3])`	True
`range(start, stop, step)`	Integer sequence, commonly used in loops.	`list(range(1, 5))`	[1, 2, 3, 4]
`enumerate(iterable, start=0)`	Pairs each item with an index.	`list(enumerate(['a', 'b']))`	[(0, 'a'), (1, 'b')]
`zip(*iterables)`	Pairs items from iterables.	`list(zip([1, 2], ['a', 'b']))`	[(1, 'a'), (2, 'b')]
`map(function, iterable)`	Applies a function to each item.	`list(map(str, [1, 2]))`	['1', '2']
`filter(function, iterable)`	Keeps items where function is truthy.	`list(filter(None, [0, 1, 2]))`	[1, 2]
`iter(obj)`	Gets an iterator.	`iter([1, 2])`	Iterator object
`next(iterator)`	Gets the next item.	`next(iter([7, 8]))`	7
`reversed(iterable)`	Returns a reverse iterator.	`list(reversed([1, 2, 3]))`	[3, 2, 1]
`slice(start, stop, step)`	Creates a slice object.	`['a','b','c'][slice(1, 3)]`	['b', 'c']
`hasattr(obj, name)`	Checks if an attribute exists.	`hasattr([], 'append')`	True
`getattr(obj, name)`	Gets an attribute by name.	`getattr('hi', 'upper')()`	'HI'

```
names = ["Anu", "Ben"]
scores = [90, 82]

for i, name in enumerate(names, start=1):
    print(i, name)

for name, score in zip(names, scores):
    print(f"{name}: {score}")

# 1 Anu
# 2 Ben
# Anu: 90
# Ben: 82
```

In Python 3, map(), filter(), zip(), reversed() and range() return iterable objects rather than lists. Use list(...) when you need to display or store all results at once.

11. Operators

Operators perform calculations, comparisons, logical tests, membership checks and bit-level operations.

Function / method	Purpose	Example	Output / notes
`Arithmetic: + - * /`	Basic arithmetic.	`10 / 4`	2.5
`Floor division: //`	Floor-divides a number.	`10 // 4`	2
`Remainder: %`	Returns the remainder.	`10 % 4`	2
`Power: **`	Exponentiation.	`2 ** 3`	8
`Comparison: == != < <= > >=`	Compares values.	`5 != 3`	True
`Logical: and / or / not`	Combines or negates truth tests.	`True and False`	False
`Membership: in / not in`	Checks containment.	`'py' in 'python'`	True
`Identity: is / is not`	Checks object identity.	`x is None`	True if x is None
`Assignment: = += -= *= /=`	Assigns or updates a variable.	`x = 2; x += 3`	x is 5
`Bitwise: & | ^ ~ << >>`	Operations on integer bits.	`5 & 3`	1
`Indexing: s[i]`	Gets an item by position/key.	`[10, 20, 30][1]`	20
`Slicing: s[a:b:c]`	Gets a portion of a sequence.	`[10, 20, 30, 40][1:3]`	[20, 30]

```
print(17 // 5)       # 3
print(17 % 5)        # 2
print(2 ** 4)        # 16
print(3 in [1, 2, 3])# True
print(5 & 3)         # 1
print(5 | 3)         # 7
```

Important: Use == to compare values. Use is for identity checks, commonly x is None. The operators and/or short-circuit, so they may skip evaluating the second operand.

12. Math module and numeric helpers

Import the math module before using its functions and constants: import math.

Function / method	Purpose	Example	Output / notes
math.sqrt(x)	Square root.	`math.sqrt(16)`	4.0
math.floor(x)	Rounds down to an integer.	`math.floor(3.7)`	3
math.ceil(x)	Rounds up to an integer.	`math.ceil(3.2)`	4
math.factorial(n)	Factorial of a non-negative integer.	`math.factorial(5)`	120
math.gcd(a, b)	Greatest common divisor.	`math.gcd(12, 18)`	6
math.trunc(x)	Removes the fractional part toward zero.	`math.trunc(-3.7)`	-3
math.fabs(x)	Absolute value as a float.	`math.fabs(-7)`	7.0
math.isfinite(x)	Checks that x is finite.	`math.isfinite(3.5)`	True
math.isclose(a, b)	Checks approximate equality.	`math.isclose(0.1 + 0.2, 0.3)`	True
math.pi	Approximation of pi (constant).	`math.pi`	3.141592653589793
math.e	Euler's number (constant).	`math.e`	2.718281828459045
abs(), round(), pow(), divmod()	Built-ins; no math import needed.	`round(2.718, 2)`	2.72

```
import math

radius = 3
area = math.pi * radius ** 2
print(round(area, 2))       # 28.27
print(math.sqrt(81))        # 9.0
print(math.gcd(24, 18))     # 6
```

Python's round() uses ties-to-even for exact halfway cases; for example, round(2.5) returns 2 and round(3.5) returns 4.

13. Random module

Use import random before calling these functions. Results can change each time the program runs.

Function / method	Purpose	Example	Output / notes
random.random()	Random float from 0.0 up to but not including 1.0.	`random.random()`	Example: 0.7312
random.randint(a, b)	Random integer including both endpoints.	`random.randint(1, 6)`	Example: 4
random.choice(seq)	Selects one item from a non-empty sequence.	`random.choice(['a','b','c'])`	Example: 'b'
random.shuffle(list)	Shuffles a list in place.	`random.shuffle(items)`	items reordered; returns None
random.sample(population, k)	Selects k unique items.	`random.sample(range(10), 3)`	Example: [7, 2, 9]
random.randrange(start, stop)	Selects from a range of integers.	`random.randrange(1, 6)`	1 to 5

```
import random

number = random.randint(1, 6)
print(number)  # Any integer from 1 through 6

items = ["red", "blue", "green"]
print(random.choice(items))  # One of the three strings
```

Use the secrets module rather than random for security-sensitive tokens or passwords. random is intended for simulations, games and ordinary randomized tasks.

14. File handling

Use open() to work with files. The with statement is generally recommended because it closes the file automatically, even if an error occurs.

Function / method	Purpose	Example	Output / notes
open(path, mode)	Opens a file and returns a file object.	`open('notes.txt', 'r')`	File object; file must exist
read(size=-1)	Reads all or part of the remaining content.	`f.read()`	String in text mode
readline()	Reads the next line.	`f.readline()`	One line as a string
readlines()	Reads remaining lines into a list.	`f.readlines()`	List of strings
write(text)	Writes text to a file.	`f.write('Hi')`	Returns number of characters written
writelines(lines)	Writes an iterable of strings.	`f.writelines(['A\n','B\n'])`	Does not add newlines automatically
seek(offset)	Moves the file position.	`f.seek(0)`	Returns new position
tell()	Reports current file position.	`f.tell()`	Integer position
close()	Closes the file.	`f.close()`	File is closed

Common file modes

Mode	Meaning	Important note
`'r'`	Read	`File must exist`
`'w'`	Write	`Creates or truncates an existing file`
`'a'`	Append	`Writes at the end; creates if needed`
`'x'`	Exclusive create	`Fails if file already exists`
`'b' / 't'`	Binary / text mode	`Text mode is the default`

```
with open("sample.txt", "w", encoding="utf-8") as f:
    f.write("Hello Python\n")
    f.write("File handling")

with open("sample.txt", "r", encoding="utf-8") as f:
    content = f.read()

print(content)
# Hello Python
# File handling
```

Common mistake: Opening a file with mode 'w' erases existing contents first. Use 'a' to append. Prefer with open(...) as f: to manage closing automatically.

15. Exception handling

Exception handling uses language statements and clauses, not just functions. It lets a program respond to expected errors without hiding unrelated bugs.

Construct	Purpose	Example pattern	Behavior
`try`	Wraps code that may raise an exception.	`try: value = int(text)`	Runs the protected code
`except ValueError`	Handles a matching exception.	`except ValueError: print('Enter a number')`	Runs when that error occurs
`else`	Runs if the try block has no exception.	`else: print(value)`	Optional success path
`finally`	Runs whether or not an exception occurs.	`finally: print('Finished')`	Cleanup path
`raise`	Raises an exception explicitly.	`raise ValueError('Invalid age')`	Stops normal flow unless handled

```
text = "abc"

try:
    number = int(text)
except ValueError:
    print("Please enter a valid integer.")
else:
    print(number)
finally:
    print("Input check complete.")

# Please enter a valid integer.
# Input check complete.
```

Best practice: Catch specific exceptions, such as ValueError, rather than using a bare except that can hide unrelated programming errors.

16. Quick reference and common mistakes

Use these tables to select a suitable tool quickly and to avoid common beginner errors.

Which tool should I use?

Goal	Typical choice
Count items	len(collection)
Add one item to a list	my_list.append(item)
Add multiple items to a list	my_list.extend(iterable)
Remove and retrieve the last list item	my_list.pop()
Find common set values	set_a.intersection(set_b) or set_a & set_b
Read a possibly missing dictionary key	my_dict.get(key, default)
Split text into words	text.split()
Join strings	'separator'.join(strings)
Convert input numbers	list(map(int, input().split()))
Sort without modifying the original	sorted(iterable)
Loop with an index and item	for i, item in enumerate(iterable):

Methods that modify the original object

Operation	Changes original?	Return behavior
list.append(), extend(), insert()	Yes	None
list.sort(), reverse(), clear()	Yes	None
list.pop()	Yes	Returns removed item
set.add(), update(), discard()	Yes	None
dict.update(), clear()	Yes	None
sorted(iterable)	No	Returns a new list
str.upper(), lower(), replace()	No; strings are immutable	Returns a new string
tuple.count(), index()	No	Returns count or index

Common mistakes

Mistake	Correct understanding
result = my_list.append(3)	The list changes; result is None.
my_list = my_list.sort()	This assigns None. Use my_list.sort() or sorted(my_list).
T = (5)	This is an integer. A one-item tuple is (5,).
S = {}	This creates a dictionary. Use set() for an empty set.
'3' + 4	Convert deliberately: int('3') + 4.
bool('False') is False	It is True because the string is non-empty.
Using is for value equality	Use == for values; use is for identity checks.
Expecting set order to be stable	Set display order is not guaranteed.

Final reminder: Use the examples as starting points, run them in Python, and adapt them to your task. When an output depends on random selection, object identity or set ordering, it may vary between runs.